How the code is organized

Open Factory Studio is a Qt 6 Widgets program made of a few libraries:

Target	Kind	What it does
ofs	program	Starts the app, loads the plugins and shows the main window
ofsshell	static library	The main window, menus, project tabs and the program's dialogs
ofscore	shared library	The base of everything: settings, actions and shortcuts, themes, Lua presets, projects, the shared editors (code, sprites, animations, sounds, palettes), drawing and audio playback
ofsformats	shared library	File formats and codecs (SFF, SND, PCX, ACT, FNT, GIF and so on). Plain C++20 without Qt
ofsaudio	shared library	Sound data: decoding and encoding WAV, Ogg Vorbis, FLAC and MP3, resampling, waveform edits. Plain C++20 without Qt
ofslua	shared library	Lua 5.4, shared by the core and all plugins
ofs_mugen	plugin	Everything for M.U.G.E.N: project types, the syntax database, its editors and tools, Run in engine and the emulator

ofs uses ofsshell, which uses ofscore. ofscore uses ofsaudio, ofsformats and ofslua (through sol2). The M.U.G.E.N plugin only talks to ofscore.

Folders

Folder	What's in it
src/formats	File formats without Qt: util, image, codecs, palette, sff, snd, wav, fnt and maker (SprMaker2 and SndMaker files). The audio folder is the separate ofsaudio library
src/core	The core library
src/shell	The main window and dialogs
src/app	The program's entry point and resources
src/engines/<name>	Engine plugins
data	Shared data: presets, icon sets, styles, color sets, help pages and the engines' own data
tests	Unit, GUI, corpus and oracle tests. tests/fixtures makes the test files in code
tools	Developer tools (ofs-sffdump and some Python scripts)
third_party	Small libraries kept in the repository (LodePNG, uchardet, miniaudio, dr_flac, dr_mp3). The rest is downloaded by cmake/OfsDependencies.cmake at fixed versions
docs	These documents

At runtime the program finds its data folder through OFS::Paths (src/core/app/Paths.h). User settings are kept in %LOCALAPPDATA%\Open Factory Studio on Windows, or the usual place on other systems.

Main ideas

- **Engines** (core/engine): an engine plugin says which project types, versions and file types it handles, and creates the editors for them. See the Engine plugins document.
- **Actions** have fixed names like file_open, spr_add or anim_play. Menus, toolbars, panels, shortcuts and presets all refer to actions by name.
- **Presets** are Lua scripts that build the toolbars and panels from named controls.
- **Projects** own their files, and every file has its own undo history. Text files keep their exact text, so the parts you didn't touch are saved byte for byte.
- **File formats** (OFS::Formats) read files with bounds checks and return errors in Result<T> values instead of throwing. The documents (SffDocument, SndDocument, FntDocument) keep the original bytes of everything you didn't edit, so saving an unchanged file gives the same file back. The byte layouts are in the File formats document.

Audio

The sound code is split in two: a library without Qt, and a small playback part in the core.

ofsaudio (src/formats/audio, namespace OFS::Formats) does everything that works on samples:

- SampleBuffer holds the samples with a channel count and a sample rate, as 16-bit integers (exact for SND sounds and half the memory) or as floats. Converting between the two uses the WAV scale and rounds carefully, so going from 16-bit to float and back loses nothing. fromWav() and toWav() convert from and to WAV files (8, 16, 24 and 32-bit PCM, 32 and 64-bit float). convertChannels() turns mono into stereo and back, and quantize() lowers the bit depth.
- AudioCodec.h: detectAudioFormat() tells WAV, Ogg Vorbis, FLAC and MP3 apart by their bytes (the file name doesn't matter). decodeAudio() decodes them, WAV files also in A-law, mu-law, IMA ADPCM, Microsoft ADPCM and MPEG. encodeAudio() writes WAV, Ogg Vorbis and FLAC. Decoding uses dr_flac, dr_mp3 and libvorbis, and encoding uses libvorbis, libogg and libFLAC. The same input always gives the same file. Decoding stops at 2^28 samples.
- Resampler.h: libsamplerate's sinc converters (Best, Medium, Fastest) plus simple Linear and ZeroOrderHold modes. The result always has round(frames × new rate / old rate) frames and stays in time with the input. resampleToLength() is the horizontal resize tool.
- AudioEdit.h: the waveform editor's operations. Each one returns a new buffer: copy, remove, trim, insert or replace (the pasted clip is converted first), insert or fill silence, fades, volume, normalise, reverse, move, stretch and finding silence. Ranges are counted in frames and clipped to the buffer.
- WaveformPeaks keeps, for each channel, the lowest and highest value of every block of 256 frames, and of bigger blocks on top of that. That gives the exact peaks of any range quickly, at any zoom. A ten-minute stereo track needs about 2 MB. It can be built at once or bit by bit (extend()), and it is updated after edits (update() when the length stays the same, otherwise invalidateFrom() and extend()).

ofsaudio is kept separate from ofsformats because its codecs and the resampler are bigger than the whole formats library, and the sprite tools and ofs-sffdump don't need any of them.

Playback (src/core/audio) uses miniaudio:

- AudioEngine (plain C++): play() opens a device with the buffer's channels and sample rate (miniaudio converts to what the hardware wants) and plays from a start frame, and can loop a range. There are also pause(), resume(), stop(), position() and setVolume(). After the last sample it plays silence for one device buffer before it reports the end, so sounds are never cut off. The System backend tries every audio API miniaudio has apart from its null device, so on a machine without sound output play() returns false and errorString() says why. The Null backend is a fake device for the tests.
- SoundPlayer (a QObject) is what the editors use. It has the same calls plus the stateChanged, positionChanged (every 30 ms while playing, for the cursor), finished and deviceLost signals.

Threads: the audio thread reads the buffer through a std::shared_ptr<const SampleBuffer>, so a buffer must not change while it plays (edits return new buffers anyway). Calls to the engine and the player come from one thread. Decoding, encoding, resampling, editing and building peaks can run on worker threads. WaveformPeaks has no locking of its own.